CASPO: Learned Prefix Value Modeling for Stepwise Credit Assignment in Reasoning RL

Anonymous

Abstract

We propose CASPO (Credit Assignment Step Policy Optimization), a method for reinforcement learning of mathematical reasoning that obtains step-level credit signals from a learned prefix-value model rather than from on-policy Monte Carlo prefix rollouts. CASPO retains the VinePPO step-segmentation and step-TD advantage structure but replaces VinePPO's K=9 rollouts at each non-terminal prefix with a single forward pass through an IPVRM-style cumulative-log-ratio prefix value model, optionally fine-tuned online during policy training under an ADB+DLW correction. The result is a strict drop-in for VinePPO that uses the same PPO clipped objective, the same KL penalty, the same response segmentation, and the same global PPO minibatch, but eliminates the per-step Monte Carlo expansion that dominates VinePPO's wall-clock cost. We compare four methods (PPO, GRPO, VinePPO, CASPO) under one trainer, one verifier, one rollout engine, and one set of hyperparameters, and report wall-clock and accuracy on the MATH benchmark for both Rho-1B and DeepSeekMath-7B. Throughout we keep the algorithmic contract identical except for the source of the per-step value $V(s_t)$, isolating the credit-signal change from confounders. Empirical results are reported in Section 6 after the corresponding training and evaluation runs complete.

1 Introduction

Reinforcement learning has emerged as the dominant approach for improving the mathematical reasoning of large language models, but the dominant signal is sparse: a verifier returns 0/1 only at the end of a long chain of thought, leaving the optimizer to assign credit across hundreds of intermediate tokens. PPO [5] addresses this by broadcasting a sequence-level advantage uniformly across response tokens; GRPO [6] refines the variance term using a group-relative baseline. Neither method distinguishes a correct intermediate inference from a lucky guess that nonetheless reached the right final answer.

VinePPO [2] addresses this gap by introducing *step-level* value estimates: the response is segmented into reasoning steps, and the value at each step boundary s_t is estimated by sampling K on-policy continuations from s_t and averaging their terminal verifier rewards. This produces well-calibrated step values that bear out empirically—VinePPO reports substantial improvements over PPO and GRPO on MATH and GSM8K—but at a cost: each non-terminal step boundary requires K additional generations, multiplying wall-clock time by roughly the average step count per response.

CASPO replaces those Monte Carlo prefix rollouts with a single forward pass through a *learned* prefix value model $V_\phi(s_t)$. Following IPVRM [1], V_ϕ is parameterized as a cumulative log-ratio between a trainable prefix policy π_ϕ and a frozen reference π_{ref}, and trained from trajectory-level correctness labels using a margin BCE objective. CASPO keeps every other VinePPO component fixed: the same LaTeX-aware step splitter, the same step-TD advantage construction, the same clipped PPO objective, and the same KL penalty. Step-level credit is therefore extracted from V_ϕ

in $\mathcal{O}(1)$ forward passes per response rather than $\mathcal{O}(K \cdot \text{steps})$ generations, and online updates of V_ϕ during policy training mitigate distribution shift.

Contributions. We contribute (i) CASPO: a method that drops VinePPO's prefix MC step in favor of an IPVRM-style prefix value model, including the step-TD construction and an online update path with ADB and DLW corrections; (ii) a unified comparison stack that implements PPO, GRPO, VinePPO, and CASPO under one trainer, one verifier, and one rollout engine, controlling everything except the algorithmic credit-signal choice; (iii) a CASPO advantage-transform ablation that probes the role of the prefix-value parameterization; (iv) a frozen-RM CASPO ablation that turns off the online value update; and (v) implementation infrastructure for full-finetune 7B-scale RL with FSDP-sharded trainer and rank-local vLLM generation, including a CUDA-IPC weight-sync path that is collective with `summon_full_params`.

Implementation provenance. The codebase is an adaptation and extension of the public VinePPO setup [2, 4]. The Rho-1B and DeepSeekMath-7B configurations, prompt templates, rollout shapes, PPO hyperparameters, evaluation protocol, and LaTeX-aware step segmentation are matched to or derived from the upstream paper and reference repository so that any wall-clock or accuracy gap between methods reflects the algorithmic change rather than infrastructure.

2 Related Work

Policy gradient with sparse rewards. PPO [5] optimizes a clipped policy-gradient surrogate to bound the per-step policy update. Applied to language models with verifier-only terminal rewards, the standard practice is to broadcast the sequence-level advantage uniformly across response tokens and add a KL-to-reference penalty to prevent the policy from drifting far from the SFT initialization.

Group-relative baselines. GRPO [6] samples G responses per prompt and uses the within-group reward statistics as the advantage baseline. This avoids training a separate value network entirely, at the cost of treating every intermediate token in a correct response as equally important.

Step-level credit assignment via Monte Carlo prefix values. VinePPO [2] introduces stepwise value estimates by sampling K on-policy continuations from each non-terminal prefix and averaging the verifier outcomes of those continuations. Step advantages are then formed as temporal-difference differences between consecutive prefix values. The reasoning-credit gain is well demonstrated on MATH, but the method's rollout multiplier scales as K times the average number of steps per response.

Learned process supervision. A line of work trains a separate process reward model on human-annotated or self-supervised step-correctness labels. CASPO's prefix value model is in this family but constructed differently: rather than scoring step text directly, it accumulates token-level $\log[\pi_\phi/\pi_{\text{ref}}]$ contributions, following IPVRM [1]. This aligns prefix scoring with the same density that generated the response and avoids requiring a separate textual annotation.

Implicit prefix value learning (IPVRM). IPVRM [1] formalizes prefix values as cumulative log-ratios of a trainable prefix policy versus a frozen reference policy, trained from trajectory-level outcome labels using a margin BCE. IPVRM's reported setting uses LoRA adapters on a

7B base; we adopt the same parameterization but perform full-model updates, with corresponding adjustments to the online learning rate.

Generation backends and weight sync. We use vLLM [3] for high-throughput rollout and prefix caching. The implementation uses CUDA-IPC weight transfer between trainer ranks and rank-local vLLM engines; for the FSDP-sharded 7B trainer, the IPC sync is implemented as a collective `summon_full_params` block over the FSDP-wrapped policy followed by per-tensor IPC handle exchange with each rank's vLLM `EngineCore`.

3 Background

3.1 Reasoning RL setup

A trajectory consists of a problem prompt x and a model response $y = (y_1, \ldots, y_T)$ generated autoregressively. A verifier $R(x, y) \in \{0, 1\}$ returns 1 if and only if the boxed final answer in y matches the ground truth. The training objective is to maximize $\mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)} R(x, y)$ subject to a KL constraint to a frozen reference policy π_{ref}, which we instantiate as the SFT initialization.

3.2 Step segmentation

We split each response into reasoning steps using the LaTeX-aware splitter of VinePPO [2, 4], which identifies natural mathematical step boundaries (display equations, aligned blocks, sentence terminators in prose). Let $0 = t_0 < t_1 < \cdots < t_S = T$ be the resulting boundary token indices and let $s_t \triangleq y_{<t}$ denote the prefix state at boundary t.

3.3 Step-TD advantage

Given a per-step value function $V(s_t)$, both VinePPO and CASPO construct the step-TD advantage

$$A(s_t) = r_t + \gamma\, V(s_{t+1}) - V(s_t), \qquad r_t = \begin{cases} R(x, y) & t = S, \\ 0 & t < S, \end{cases} \tag{1}$$

with $\gamma = 1$. CASPO obtains V from a learned model V_ϕ; VinePPO obtains V by averaging K Monte Carlo continuations. The downstream PPO objective and KL penalty are identical for both methods; only the source of V differs.

3.4 PPO objective

We use the clipped PPO objective

$$\mathcal{L}_{\text{clip}}(\theta) = -\,\mathbb{E}_t\big[\min\big(\rho_t(\theta)\, A_t,\ \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\, A_t\big)\big]\,, \tag{2}$$

with $\rho_t(\theta) = \pi_\theta(y_t \mid x, y_{<t}) / \pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})$ and $\epsilon = 0.2$. A KL-to-reference term is added with the k_3 estimator:

$$\text{KL}_{k_3}(\pi_\theta \,\|\, \pi_{\text{ref}}) \approx \mathbb{E}_t[\exp(\delta_t) - \delta_t - 1]\,, \quad \delta_t = \log \pi_{\text{ref}}(y_t) - \log \pi_\theta(y_t). \tag{3}$$

The advantages A_t in (2) are (a) terminal-broadcast for PPO/GRPO and (b) the step-TD advantages of (1) broadcast over the tokens in each step span for VinePPO/CASPO.

4 Method: CASPO

4.1 Prefix value model

Following IPVRM [1], we define

$$V_\phi(s_t) \;=\; \beta \sum_{i<t} \log \frac{\pi_\phi(y_i \mid x, y_{<i})}{\pi_{\text{ref}}(y_i \mid x, y_{<i})}, \tag{4}$$

where π_ϕ is a trainable prefix policy initialized from the same SFT checkpoint family as the base policy and π_{ref} is the same frozen reference used in the KL term in (3). The scale β is a fixed scalar; we use $\beta = 10$ throughout.

4.2 Phase-1: offline value training

We train V_ϕ on a fixed corpus of (prompt, response, outcome) triples collected by sampling G responses per prompt from the SFT policy at generation temperature 1.0, scoring with the verifier, and discarding prompts whose G outcomes are all 0 or all 1 (the mixed-outcome filter, following IPVRM). The training loss is a margin-BCE between paired correct/incorrect prefixes:

$$\mathcal{L}_{\text{IPVRM}}(\phi) = \mathbb{E}_{(x,y^+,y^-)}\big[\log\big(1 + \exp(-(V_\phi(y^+) - V_\phi(y^-) - m))\big)\big], \tag{5}$$

with margin $m = 5$. We train for three epochs with full-model updates and a small learning rate (5×10^{-7}).

4.3 Phase-2: step-TD policy optimization

At each PPO outer step, the trainer samples G responses per prompt for P prompts, segments each response, and constructs step values $\{V_\phi(s_t)\}$ in a single forward pass through V_ϕ using (4). Step advantages are then assembled from (1) and broadcast over the tokens within each step span before applying the PPO objective in (2) with KL penalty in (3). The standard CASPO run optionally co-trains V_ϕ during policy optimization (next section).

4.4 Online value updates with ADB+DLW

Distribution drift between the SFT-init prefix policy and the actively optimized policy can degrade V_ϕ. We therefore allow online updates of V_ϕ from the same rollout batch used for PPO. The on-policy update uses the same margin-BCE loss (5) but with two corrections from IPVRM:

- *ADB* (advantage debias): the BCE boundary is shifted per-prompt by $\log\big[\mu/(1-\mu)\big]$, where μ is the empirical positive fraction over the prompt's G rollouts, so the value model learns *deviations* from the prompt's SFT difficulty rather than its absolute difficulty.
- *DLW* (difficulty-level weighting): the BCE term is weighted by $\mu(1-\mu)$, downweighting prompts where outcomes are nearly all-correct or all-incorrect.

The online value learning rate is set to 10^{-6}, an order of magnitude below the offline rate to control drift.

4.5 Advantage-transform ablation

A subtle modeling choice in CASPO is *which* statistic of V_ϕ should enter the step-TD difference. The three natural choices arise from how the IPVRM loss (5) parameterizes correctness. Recall that the BCE margin loss treats V_ϕ as a Bradley–Terry score: the implied per-prefix correctness probability is

$$\widetilde{p}(s_t) \triangleq \sigma(V_\phi(s_t)) \approx \mathbb{P}[R(x,y) = 1 \mid s_t], \tag{6}$$

where σ is the logistic function. Equation (6) is the natural way to convert an unbounded cumulative-log-ratio score into a calibrated probability, and it is the quantity the BCE objective directly optimizes.

Given this, the step-TD advantage in (1) can be computed on three different scales:

value (raw cumulative log-ratio):

$$A^V(s_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{7}$$

prob (Bradley–Terry probability):

$$A^P(s_t) = r_t + \gamma \widetilde{p}(s_{t+1}) - \widetilde{p}(s_t) = r_t + \gamma \sigma(V_\phi(s_{t+1})) - \sigma(V_\phi(s_t)). \tag{8}$$

logprob (log Bradley–Terry probability):

$$A^{LP}(s_t) = r_t + \gamma \log \widetilde{p}(s_{t+1}) - \log \widetilde{p}(s_t). \tag{9}$$

In all three variants, the terminal verifier reward $r_T = R(x,y)$ enters identically; the only difference is the function applied to V_ϕ before the TD difference.

Intuition. The three transforms have different scales and different bias–variance profiles:

- *value*: The TD operates on the unbounded log-ratio scale. A single token whose log-ratio swings by, say, -10 produces an unbounded ΔV_ϕ, which carries a strong gradient signal but also amplifies miscalibrated regions of the prefix value model (e.g. tokens far from the ref-policy distribution). High signal-to-noise where V_ϕ is well-calibrated, but heavier tails when it is not. Standardization + clipping (Section 4, advantage clip ± 3) bounds the magnitude empirically, but does not reshape the underlying scale.
- *prob*: The TD operates on the probability scale, so $|A^P(s_t)| \leq 1$ before standardization. By (6), $\widetilde{p}(s_t) \approx \mathbb{E}[R \mid s_t]$, so A^P is the empirical advantage that an idealized binary-reward critic would compute: it is the change in the expected terminal reward attributed to step t. This is the variant most directly compatible with the binary verifier we use, but it suffers *near saturation*: when $V_\phi(s_t)$ is large positive or negative, $\sigma'(V_\phi)$ collapses toward zero, so two prefixes with very different log-ratios get essentially the same gradient. In practice this means *prob* undersignals on prefixes the value model is highly confident about.
- *logprob*: The TD operates on the log-probability scale, mirroring the form of token log-likelihoods used in the PPO ratio $\rho_t(\theta)$. For a sequence model trained by maximum likelihood, log-probability differences are the canonical update signal; using $\log \widetilde{p}$ keeps the value-derived advantage on the same information-theoretic scale as the underlying token logits. Numerically, $\log \sigma(V) = -\log(1 + e^{-V})$, which behaves linearly for $V \ll 0$ (matching the value scale on the negative tail) and saturates softly to zero for $V \gg 0$ (compressing the high-confidence positive tail). This is a smooth interpolation between *value* (linear in the negative regime) and *prob* (bounded above).

When each variant should help. If V_ϕ is well-calibrated as a Bradley–Terry score, *prob* is the mathematically natural choice and admits a clean expected-reward interpretation. If V_ϕ is well-trained but operates over a wide log-ratio range that the BCE objective does not directly bound, *value* preserves more of the underlying signal but pays in variance. *logprob* is a middle ground: it preserves linear sensitivity in the negative tail (where most gradient signal lives early in training) while compressing the positive tail (where over-confident prefixes would otherwise dominate the batch). Section 6 reports the three variants on MATH-500 pass@16 to determine which scale matches the empirical calibration of V_ϕ best for our setup.

4.6 Step advantage normalization and clipping

Let $\mathcal{A} = \{A(s_t)\}$ over all valid step boundaries in the outer PPO step. We standardize over the entire batch ($\widetilde{A}_t = (A_t - \mu_{\mathcal{A}})/(\sigma_{\mathcal{A}} + \varepsilon)$) and clip the standardized advantages to $[-3, 3]$ to bound the contribution of single-step outliers. The clip range is chosen empirically to balance suppression of pathological prefixes against retaining genuine step-level signal.

5 Experimental Setup

5.1 Models and data

We evaluate on two policy initializations and one shared dataset family:

- **Rho-1B**: `realtreetune/rho-1b-sft-MATH`, a 1.1B-parameter SFT-on-MATH initialization used in the upstream VinePPO paper.
- **DeepSeekMath-7B**: `realtreetune/deepseekmath-7b-sft-MATH-v2`, the 7B SFT-on-MATH initialization from the same upstream pipeline.

Training prompts are drawn from `DigitalLearningGmbH/MATH-lighteval` (train split). Held-out evaluation uses `HuggingFaceH4/MATH-500` (test split, 500 problems) plus the full MATH test set, College Math, and OlympiadBench (math-only configuration). All settings use the prompt template `[MATH_TASK] Problem:\n{query}\n\nSolution:` and the LaTeX-aware step splitter from the upstream codebase.

5.2 Methods

We compare four methods:

PPO Terminal-reward, sequence-level advantages broadcast across response tokens.

GRPO Per-prompt group-relative terminal-reward advantages with $G = 8$.

VinePPO $K = 9$ Monte Carlo continuations from each non-terminal step boundary; step-TD advantages.

CASPO IPVRM prefix value model with online updates; step-TD advantages over V_ϕ.

We additionally report two CASPO ablations: *advantage-transform prob/logprob* and *frozen-RM* (no online value updates).

Table 1: Shared training hyperparameters (Rho-1B and 7B). All methods use the same PPO clipped objective, KL coefficient, group size, response budget, and global PPO minibatch.

Field	Rho-1B	DeepSeekMath-7B
Group size G	8	8
Prompts per outer step	64	8
Responses per outer step	512	64
Global PPO minibatch	64	64
PPO epochs per rollout	2	2
Policy LR	1×10^{-6}	2×10^{-6}
Online value LR (CASPO)	1×10^{-6}	1×10^{-6}
Value β, m	10, 5	10, 5
PPO clip ϵ	0.2	0.2
KL coefficient	10^{-4} (k_3)	10^{-4} (k_3)
Advantage clip	3.0	3.0
Advantage standardization	batch	batch
Rollout sampling	temp 0.6, top-p 0.9	temp 0.6, top-p 0.9
Max prompt / response	1024 / 1024	1024 / 1024
Outer steps	1000	1500
Checkpoints	step_250/500/750/final	step_250/500/750/1000/1250/final

5.3 Hyperparameters

All four methods share the same PPO infrastructure. Table 1 summarizes the key knobs.

The 7B run uses 1.5× the outer-step budget of the Rho-1B run because the 7B base policy needs more updates per equivalent training signal. The larger Rho-1B prompt batch reflects its lower per-step compute cost; both configurations land at a 64-response global PPO minibatch.

5.4 Evaluation protocol

Final evaluation reports pass@1 and pass@k ($k = 16$) on MATH-500, full MATH test, College Math, and OlympiadBench. Sampling at evaluation uses temperature 0.35 and top-p 0.9, matching the upstream protocol. Generation is performed with vLLM at `kv_cache_dtype="fp8"` and `gpu_memory_utilization=0.92` for maximum eval throughput on a dedicated GPU. Sample evaluation at intermediate checkpoints uses `EVAL_LIMIT=100`, `EVAL_K=8` for low-overhead training-curve visibility; the full benchmark suite is run at the final checkpoint.

5.5 Infrastructure

Both scales share a single PyTorch trainer plus rank-local vLLM rollout. The 1B configuration uses a single H100 per method; the 7B configuration uses FSDP=4 (full_shard, use_orig_params) for PPO/GRPO/CASPO and FSDP=8 for VinePPO, with vLLM colocated rank-local in both layouts. Trainer-to-vLLM weight synchronization uses CUDA IPC: at 1B with a single-rank trainer, the IPC handle exchange is direct; at 7B under FSDP, the trainer enters `FSDP.summon_full_params(writeback=False)` collectively before issuing per-tensor IPC pushes through each rank's vLLM `EngineCore.collective_rpc("update ...)`.

For numerical stability under the FSDP layout, the trainer co-trains policy and (for CASPO) value model in bf16 with fp32 master weights; the reference policy is shared between the KL

term and CASPO's value-model ref to avoid a duplicate forward. Rollout, old-logprob rescore, ref-logprob precompute, and policy/value forward use FlashAttention-3 on Hopper.

6 Results

This section will report wall-clock and accuracy after the full training and evaluation runs complete. The intended structure is:

- Per-method MATH-500 pass@1 and pass@16 at `final`, with seed-level error bars over a small number of seeds (target: 3 seeds per method per scale).
- Per-method full-suite (MATH, College Math, OlympiadBench) pass@16 at `final`.
- Training curves on MATH-500 sample evaluation at `step_250`, `step_500`, `step_750`, and `final`, summarizing sample efficiency.
- CASPO advantage-transform ablation (*value* vs. *prob* vs. *logprob*) on MATH-500 pass@16.
- CASPO frozen-RM ablation (online value updates on/off).
- Wall-clock summary: per-method step time, per-method 1500-step ETA, and the implied GPU-hour cost of the four-method comparison.

We report all numbers under the same training contract: identical SFT initialization, identical prompt set, identical verifier, identical group/response budget, identical PPO clipped objective, identical KL coefficient, identical advantage standardization, identical max sequence length, and identical evaluation sampling parameters. The only intended difference between rows is the source of the per-step value $V(s_t)$ in (1): zero (PPO/GRPO terminal broadcast), Monte Carlo continuations (VinePPO), or learned prefix value V_ϕ (CASPO).

7 Analysis

This section will examine, given the empirical results, what determines when V_ϕ matches VinePPO's prefix-rollout signal and when it does not. The intended structure is:

- *Step-credit calibration*: do step advantages from V_ϕ rank correct vs. incorrect step continuations consistently with on-policy MC?
- *Online value drift*: does the online update preserve the phase-1 calibration, or does it collapse to constant predictions? We monitor $\bar{V}^+$ and $\bar{V}^-$ (mean prefix value at correct vs. incorrect step continuations) over outer steps to detect drift.
- *Wall-clock per accuracy point*: the practical question CASPO is asked to answer is whether learned prefix values can match VinePPO's accuracy at a fraction of the wall-clock cost. We report this as accuracy-versus-GPU-hours for each method.
- *Step-count effects on VinePPO*: VinePPO's wall-clock scales as $K \cdot$ steps/response, so the marginal cost of step-level credit grows with response length. We quantify this by binning the test set into long vs. short reasoning regimes and reporting the per-bin accuracy/wall-clock trade-off.

8 Limitations

Reward-model drift. CASPO's online value updates are gated by ADB and DLW corrections, but no correction perfectly defends against pathological feedback loops (e.g. a value head that learns to predict the policy's surface form rather than its correctness). The frozen-RM ablation isolates this risk.

Segmenter quality. The LaTeX-aware step splitter is a structural heuristic. Manual spot checks on Rho-1B MATH generations show coherent boundaries for prose, equations, factorization, and aligned derivations, but generations that continue past a final boxed answer have those trailing fragments treated as additional steps; a post-answer truncation pass is left as future work.

Apples-to-apples scope. Our comparison fixes the SFT initialization, prompt set, verifier, sampling parameters, KL constraint, and advantage standardization. It does not exhaustively explore method-specific tuning (e.g. different K for VinePPO, different β/m for CASPO, larger G for GRPO); we report the upstream-paper values where available and otherwise the values that minimize wall-clock without regressing pass@1 on a small validation set.

Scale. Results at 1B and 7B may not transfer to substantially larger scales. The 7B FSDP+vLLM-IPC stack is novel to this work; whether the same configuration extrapolates to 70B is an open systems question.

9 Conclusion

We described CASPO, a step-level credit assignment method for reasoning RL that replaces VinePPO's on-policy Monte Carlo prefix rollouts with an IPVRM-style learned prefix value model and an optional online value update. CASPO fits cleanly into the same PPO clipped objective, the same KL constraint, and the same step-TD advantage construction used by VinePPO, isolating the credit-signal change from confounders. By unifying PPO, GRPO, VinePPO, and CASPO under one trainer, one verifier, and one rollout engine, we make it possible to compare these four methods under identical conditions on Rho-1B and DeepSeekMath-7B. Empirical results in Section 6 will quantify whether the learned value model recovers VinePPO's reasoning-credit signal at a fraction of its generation cost, or whether step-level Monte Carlo remains uniquely advantageous.

References

[1] Shiping Gao, Hongzhan Chen, Xiaojun Quan, Qifan Wang, and Lifu Huang. Unleashing implicit rewards: Prefix-value learning for distribution-level optimization. *arXiv preprint arXiv:2604.13197*, 2026. URL `https://arxiv.org/abs/2604.13197`.

[2] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024. URL `https://arxiv.org/abs/2410.01679`.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large

language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

[4] McGill NLP. Vineppo codebase. https://github.com/McGill-NLP/VinePPO, 2024. Code for VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.